

ParaPM: Efficient Hardware Accelerator for Postquantum Signature With High-Performance Polynomial Multiplier

Yumeng Zhao^{ID}, Gaoming Du^{ID}, Xiaolei Wang, Yongsheng Yin^{ID}, *Member, IEEE*, Duoli Zhang^{ID},
and Zhenmin Li^{ID}

Abstract—During the Institute of Standards and Technology (NIST) postquantum cryptography standardization process, the lattice-based Dilithium scheme was selected as one of the three third-round finalists for digital signature algorithms. Although numerous hardware implementations of Dilithium have been proposed, there remains substantial room for performance optimization, particularly in terms of computational speed. In this work, we target the two most time-consuming operations, namely the coefficient generation and polynomial multiplication. We propose a high-speed hardware architecture that fully exploits hardware parallelism. Our design introduces a fully pipelined radix-2 multipath delay commutator (R2MDC) structure supporting both NTT and inverse NTT (INTT) modes, a throughput-matched polynomial multiplier enabling on-the-fly pointwise multiplication (PWM) without buffering, and a low-resource Keccak. These optimizations collectively eliminate the throughput mismatch bottleneck and maximize hardware utilization through task-level parallelism. We implement coefficient generation, signing, and verification for three security levels on the A7 and Z7 platforms. Experimental results demonstrate that our current design achieves the optimal area-time product (ATP) across all three security levels.

Index Terms—Coefficient generation, CRYSTALS-Dilithium, field-programmable gate array (FPGA), hardware, high speed, polynomial multiplier, postquantum cryptography.

I. INTRODUCTION

WITH the rapid development of quantum computers, Accenture predicts that by 2028, Shor’s algorithm [1], [2] will be deployed on quantum computing platforms, signaling that existing cryptographic systems will no longer be sufficient to protect data security [3]. In response, the National Institute of Standards and Technology (NIST) began a process to select postquantum cryptographic algorithms in 2017 [4]. By 2022, NIST had standardized four algorithms—Dilithium, Falcon, SPHINCS+, and Kyber [5]—and in March 2025, HQC was announced as the fifth postquantum algorithm for encryption [6].

Received 16 December 2025; revised 18 February 2026 and 18 March 2026; accepted 18 March 2026. This work was supported in part by Hefei University of Technology under Grant W2025JSKF0152. (Corresponding authors: Zhenmin Li; Xiaolei Wang.)

The authors are with the School of Microelectronics, Hefei University of Technology, Hefei 230009, China (e-mail: zhenmin.li@hfut.edu.cn; wangxiaolei@hfut.edu.cn).

Digital Object Identifier 10.1109/TVLSI.2026.3678026

However, efficient hardware implementation of Dilithium remains a significant challenge. Computational profiling reveals that the signature-generation phase is the primary bottleneck, accounting for approximately 74% of the total execution time. This latency is dominated by the high-frequency polynomial multiplications required inside the rejection-sampling loop. Specifically, Dilithium involves intensive matrix–vector operations where a single polynomial vector is sequentially multiplied by a matrix.

Despite the urgency, existing hardware architectures fail to fully exploit the potential parallelism in this process. The fundamental limitation lies in the “tightly coupled” design of the number theoretic transform (NTT) and pointwise multiplication (PWM) units. In conventional designs, these units are often scheduled sequentially or matched with static throughput ratios. This leads to two critical inefficiencies as follows.

- 1) *Underutilized Parallelism*: Software-oriented workflows serialize the NTT and PWM steps, ignoring the field-programmable gate array (FPGA’s) capability for deep pipelining.
- 2) *Throughput Mismatch*: The volume of PWMs in the rejection loop vastly exceeds the number of required NTT operations. Existing designs often lack a mechanism to balance these unequal workloads, causing either the NTT unit to stall or the PWM unit to become a bottleneck, resulting in significant buffering overhead and structural inefficiency.

To bridge this gap, this work proposes a throughput-matched hardware architecture that harmonizes the algorithmic workflow with hardware acceleration. Building on our prior component-level work [7], which validated a pipelined radix-2 multipath delay commutator (R2MDC) polynomial multiplier, we extend this foundation to a complete, system-level implementation of Dilithium. Our design features a decoupled datapath that enables “on-the-fly” PWM, effectively eliminating the throughput mismatch and minimizing the latency of the critical rejection loop.

In our prior work [7], we proposed a fully pipelined R2MDC structure for high-speed processing, enabling concurrent support for both NTT and inverse NTT (INTT) modes, and a fully pipelined polynomial multiplication architecture that overlaps PWM with NTT/INTT. At Security Level V, the design reduces latency by 33% relative to the fastest prior

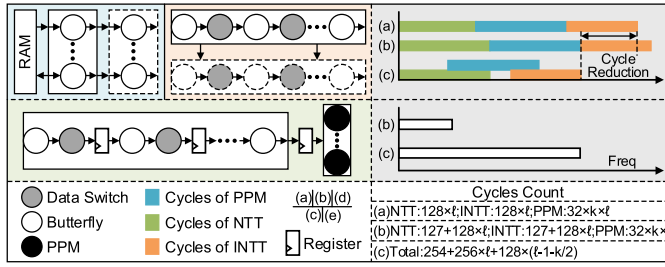


Fig. 1. Motivation example. (a) Memory-based architecture employs reused BF units for NTT/INTT and PWM operations, leading to extended cycles. (b) Sequential connection of BF units results in reduced clock frequency for pipeline-based configuration. (c) This work integrates an independently developed PWM unit to shorten cycle times, and pipeline structure enhancements increase clock frequency. (d) Cycle reduction. (e) Comparison of the fundamental frequencies between the two structures.

design and improves area-time product (ATP) by 20% over top benchmarks (Fig. 1). Building on these foundations, our work extends the high-performance polynomial multiplier to a complete hardware implementation of the Dilithium digital-signature algorithm.

A. Related Work

Postquantum cryptography spans lattice-based, code-based, multivariate-based, and hash-based schemes. Hardware implementations of Dilithium, a lattice-based signature scheme reliant on M-LWE [8], and SIS [9] problems, have been actively researched. Existing works can be broadly categorized into resource-intensive parallel designs, compact co-designs, and algorithmic optimizations.

Early implementations prioritized throughput by instantiating massive parallelism. For instance, Beckwith et al. [10] employed three concurrent KECCAK units and multiple sampling units to accelerate coefficient generation. Similarly, Ricci et al. [11] utilized independent hardware modules for SHAKE128/256 and matrix expansion. However, this approach incurs prohibitive hardware resource overhead, making deployment on resource-constrained edge devices impractical.

To optimize area efficiency, recent works have shifted toward hardware-software co-design. Wang et al. [12] proposed a compact architecture using a single KECCAK unit, where matrix A and vectors s_1, s_2 are generated sequentially. Mao et al. [13] adopted a unified sampler to process data for rejection sampling. While these designs improve integration, they often suffer from limited parallelism. For example, the sequential generation of polynomial coefficients in [12] creates a long latency path as vector dimensions grow. Similarly, in [13], the overall latency remains high (at least 168 cycles for permutation) due to the serialization of data dependencies.

Several works focus on optimizing specific arithmetic kernels. Zhao et al. [14] reduced the KECCAK permutation to 12 cycles, yet the parallelism of the rejection sampling remained low. More recently, Li et al. [15] proposed a multi-instruction architecture to partition processes into parallel instructions, and Zhao et al. [16] introduced a sparse multiplication technique to reduce bit-level complexity. Cui et al. [17] further refined the modular multiplication scaling and opti-

mized the radix-2 MDC-NTT architecture to reduce BRAM usage. Bisheh-Niasar et al. [18] proposed a parametric NTT design that improved efficiency by 44% with over 25% fewer Slices and 80% fewer BRAMs. Consequently, they realized a compact and high-speed ASIC implementation within the same year [19].

Despite these advancements, a critical gap remains in system-level scheduling. Most state-of-the-art designs focus on optimizing individual modules (NTT or Keccak) in isolation but overlook the throughput mismatch between the coefficient generation and the intensive polynomial multiplication in the signing phase. They either couple these stages too tightly (causing stalls) or serialize them (increasing latency). There is a lack of a unified architecture that dynamically matches the throughput of NTT and PWM to fully exploit task-level parallelism without excessive resource consumption. This work aims to address this specific limitation.

B. Contribution

This work combines our previously reported high-performance polynomial multiplier architecture (Du et al. [7]) with the development of a complete Dilithium hardware architecture. The main contributions of this work can be summarized as follows.

- 1) We design a fully pipelined R2MDC NTT/INTT architecture incorporating a five-stage pipelined butterfly (BF) unit, which supports both NTT and INTT. By inserting strategically placed pipeline registers and replacing the register-based sort buffer with BRAM, the proposed design mitigates the frequency degradation caused by the long combinational paths in traditional serial BF connections, thereby achieving high clock rates without incurring excessive resource overhead.
- 2) We decouple the NTT and PWM into independent modules, enabling true parallel execution. The proposed throughput-matching multiplier performs point-wise operations immediately once the NTT outputs are ready, thereby eliminating the need to store the NTT results. To execute PWM, four configurable processing elements (PEs) are instantiated, allowing the NTT outputs to be processed concurrently with multiple vectors. This design achieves dynamic throughput matching according to computational demands.
- 3) Building upon the parallel polynomial multiplier in (2), we restructure the software workflow of the Dilithium algorithm to enable concurrent execution of independent polynomial multiplications (e.g., $c \times s_1$, $c \times s_2$, and $c \times t_0$). This approach eliminates unnecessary serialization, with the most significant improvement observed in the sign stage, where it reduces the rejection loop delay and shortens the overall signature computation time by 20%–40%.
- 4) We introduce mathematical factorization techniques for large-constant multiplications, breaking down 28×23 -bit operations into smaller DSP-friendly units, and using shift-and-add operations for selected twiddle factors. These optimizations reduce DSP usage while improving maximum operating frequency.

- 5) We optimize the SHA-3 Keccak unit by eliminating redundant XOR operations unnecessary for hardware execution, leading to significant logic resource savings without affecting functionality or speed.

II. PRELIMINARIES

A. Crystals-Dilithium Overview

Dilithium uses a rejection sampling approach within a lattice-based Fiat–Shamir framework derived from MLWE and employs Gaussian sampling to produce signatures. Among lattice-based signature schemes that rely solely on uniform sampling, Dilithium offers the smallest public-key and signature sizes. The key generation, signing, and verification procedures are described in [20, Algorithms 1–3].

Algorithm 1 Key Generation

```

01:  $\xi \leftarrow \{0, 1\}^{256}$ ; ▷ Generate a 256-bit random number( $\xi$ )
02:  $(\rho, \zeta, k) \in \{0, 1\}^{256 \times 3} = \text{SHAKE256}(\xi)$ ;
03:  $(s_1, s_2) \in S_\eta^\ell \times S_\eta^k = \text{SHAKE128}(\zeta)$ ;
04:  $A \in \mathbb{R}^{k \times \ell} = \text{SHAKE128}(\rho)$ ;
05:  $t = A \times s_1 + s_2$ ;
06:  $(t_1, t_0) = \text{Power2Round}_q(t, d)$ ;
07:  $\text{tr} \in \{0, 1\}^{384} = \text{CRH}(\rho \parallel \text{tr})$ ; ▷ CRH is instantiated as
    SHAKE256
08: return(pk =  $(\rho, t_1)$ , sk =  $(\rho, k, \text{tr}, s_1, s_2, t_0)$ );

```

Algorithm 2 Signing

```

01:  $A \in \mathbb{R}_q^{k \times \ell} := \text{ExpandA}(\rho)$ 
02:  $\mu \in \{0, 1\}^{384} := \text{CRH}(\text{tr} \parallel M)$ ;
03:  $\kappa = 0, (\mathbf{z}, \mathbf{h}) := \perp$ ;
04:  $\rho' \in \{0, 1\}^{384} := \text{CRH}(K \parallel \mu)$ ;
05: while  $(\mathbf{z}, \mathbf{h}) = \perp$  do;
06:  $\mathbf{y} \in \tilde{S}_{\gamma_1}^1 := \text{ExpandMask}(\rho', \kappa)$ ;
07:  $\mathbf{w} := A\mathbf{y}$ 
08:  $\mathbf{w}_1 := \text{HighBits}_q(\mathbf{w}, 2\gamma_2)$ ;
09:  $\tilde{c} \in \{0, 1\}^{256} := H(\mu \parallel \mathbf{w}_1)$ ;
10:  $c \in B_\tau := \text{SampleInBall}(\tilde{c})$ ;
11:  $\mathbf{z} := \mathbf{y} + c\mathbf{s}_1$ ;
12:  $\mathbf{r}_0 := \text{LowBits}_q(\mathbf{w} - c\mathbf{s}_2, 2\gamma_2)$ ;
13: if  $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$  or  $\|\mathbf{r}_0\|_\infty \geq \gamma_2 - \beta$  then;
14:  $(\mathbf{z}, \mathbf{h}) := \perp$ ;
15: else
16:  $\mathbf{h} := \text{MakeHint}_q(-c\mathbf{t}_0, \mathbf{w} - c\mathbf{s}_2 + c\mathbf{t}_0, 2\gamma_2)$ ;
17: if  $\|c\mathbf{t}_0\|_\infty \geq \gamma_2$  or the # of 1's in  $\mathbf{h}$  is greater than  $\omega$  then
18:  $(\mathbf{z}, \mathbf{h}) := \perp$ 
19: end if
20: end if
21:  $\kappa := \kappa + 1$ 
22: end while
23: return  $\sigma = (\mathbf{z}, \mathbf{h}, \tilde{c})$ 

```

Table I summarizes the parameters across the three NIST security levels. The modulus q , polynomial degree n , and the number of dropped bits from t remain fixed across levels. In contrast, the dimension of the matrix A increases with higher security levels, reflecting stronger security guarantees

Algorithm 3 Verification

```

01:  $A \in \mathbb{R}_q^{k \times \ell} := \text{ExpandA}(\rho)$ 
02:  $\mu \in \{0, 1\}^{384} := \text{CRH}(\text{CRH}(\rho \parallel \mathbf{t}_1) \parallel M)$ ;
03:  $c := \text{SampleInBall}(\tilde{c})$ ;
04:  $\mathbf{w}'_1 := \text{UseHint}_q(\mathbf{h}, A\mathbf{z} - c\mathbf{t}_1, 2\gamma_2)$ ;
05: if  $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$  and  $\tilde{c} = H(\mu \parallel \mathbf{w}'_1)$  and # of 1's in  $\mathbf{h}$  is  $\leq \omega$ 
    then
06: return Valid;
07: else;
08: return Invalid;
08: end if;

```

and greater computational complexity. This parameter scaling also results in changes to η and γ_1 . The generation of A incurs substantial latency, and producing the column vectors $\mathbf{s}_1$ and $\mathbf{s}_2$ requires a nontrivial number of cycles. Consequently, the efficient generation of polynomial coefficients becomes a critical factor in optimizing overall performance.

B. Number Theoretic Transform

NTT [21], [22] serves as the number-theoretic counterpart to the discrete Fourier transform (DFT) [23], [24]. Among existing methodologies, both the fast Fourier transform (FFT) [25] and the NTT achieve the optimal asymptotic time complexity of $O(n \log n)$. However, unlike the FFT, which relies on floating-point arithmetic and consequently suffers from precision loss, NTT operates within the integer coefficient polynomial ring $\mathcal{R}_q = \mathbb{Z}_q[x]/(x^N + 1)$. This reliance on exact integer arithmetic has established NTT as the predominant approach for contemporary polynomial multiplication. In the context of lattice-based cryptography, polynomial multiplication over the ring is typically implemented via negative wrapped convolution (NWC). This process involves transforming the polynomials via NTT, performing point-wise multiplication, and reconstructing the result via the INTT. Formally, the NTT is defined as $\hat{a}_i = \sum_{j=0}^{N-1} a_j \omega^{ij} \bmod q$, and the INTT is expressed as $a_i = N^{-1} \sum_{j=0}^{N-1} \hat{a}_j \omega^{-ij} \bmod q$.

Two standard BF structures are used to implement NTT/INTT: the Cooley–Tukey (CT) BF and the Gentleman–Sande (GS) BF. The CT BF follows a decimation-in-time (DIT) approach, taking inputs in standard order and producing outputs in bit-reversed order. The GS BF follows a decimation-in-frequency (DIF) approach, taking inputs in bit-reversed order and producing outputs in standard order. Combining CT for the forward NTT and GS for the INTT eliminates the need for explicit bit-reversal.

III. PROPOSED ARCHITECTURE

A. Top-Level Architecture of Dilithium

1) *Timing Scheduling*: To illustrate how the proposed architecture improves parallelism, the signing-phase timing schedule is shown in Fig. 2. Our optimization targets the polynomial multiplications inside the rejection loop. In the traditional design, after the NTT, the polynomial $\mathbf{c}$ is multiplied sequentially with the independent polynomials $\mathbf{s}_1$, $\mathbf{s}_2$, and $\mathbf{t}_0$ in a strictly serial fashion. However, $\mathbf{s}_1$, $\mathbf{s}_2$, and $\mathbf{t}_0$

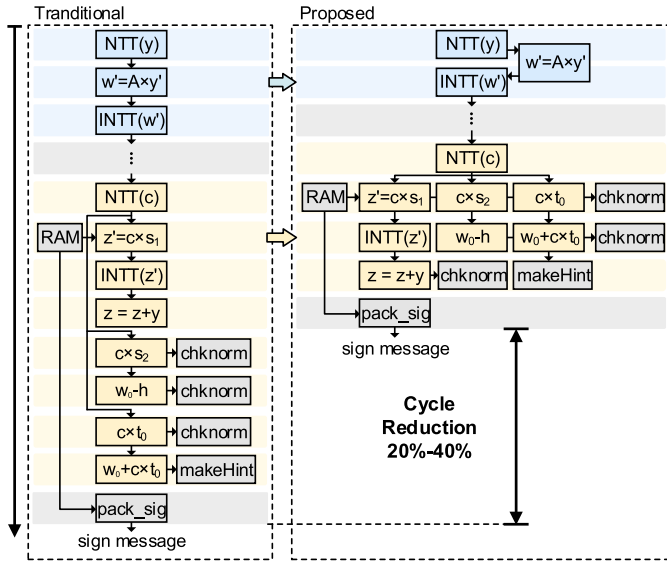


Fig. 2. Flowchart of the optimized signing stage, demonstrating a 40% reduction in clock cycles compared with conventional implementations.

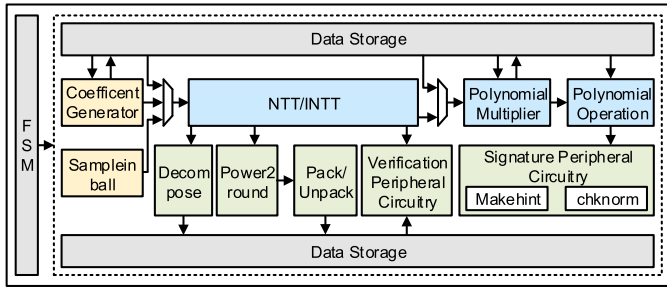


Fig. 3. Hardware architecture of Dilithium.

are all precomputed and mutually independent, permitting their corresponding multiplications to be executed in parallel. Although this improvement may appear modest, it yields a substantial performance gain: our experimental results indicate that this optimization translates to a 20% to 40% reduction in the overall signing latency across different security levels. This improvement becomes increasingly pronounced at higher security levels, where vector dimensions are larger, and the rejection probability necessitates more frequent loop executions.

2) *Architecture of Dilithium*: The overall hardware architecture of the Dilithium implementation is illustrated in Fig. 3, and comprises the following components: a finite state machine (FSM), coefficient generation unit, polynomial multiplier, coefficient operation unit, signature peripheral circuitry, verification peripheral circuitry, Power2Round, compression/decompression unit, and data storage.

The FSM serves as the status control module, coordinating all functional units. The coefficient generator (CoeGen) supplies the random values required at each stage of the algorithm. The polynomial multiplier performs all polynomial multiplication tasks. The coefficient operation unit primarily carries out polynomial addition and subtraction following the INTT stage. The data storage module holds intermediate results produced by various computational units.

The signature peripheral circuitry handles operations such as mask generation, MakeHint, rejection loop, SampleInBall, index decomposition, and packing. The verification peripheral circuitry manages timing and control flow for verification operations, including value comparisons. The Power2Round module remaps out-of-range data back into the modular domain while compressing the public-key element t to improve hardware area efficiency. The compression/decompression unit compresses the public key during key generation and decompresses it during signing and verification.

B. Polynomial Multiplier

To efficiently execute the parallel scheduling strategy proposed in Section III-A, the hardware architecture must support high-throughput, decoupled operations. We adopt the high-performance R2MDC NTT kernel and the pipelined BFU unit (BFU) structure from our previous work [7]. This kernel provides the fundamental arithmetic capability for separate NTT and PWM operations.

Polynomial multiplication consists of two main components: NTT and PWM. Most existing designs integrate these two functions, reusing DSP resources within the NTT to perform polynomial point-wise multiplication. In contrast, we separate these functions and design them independently. This approach is motivated by the following reasons: when restructuring the software flow, we parallelize the triple polynomial point-wise multiplication. To meet this parallelization requirement, the PWM module must have sufficient resources; relying solely on the internal resources of the NTT module is insufficient. Additionally, introducing dedicated PWM modules can further increase the polynomial multiplication throughput.

To finalize the NTT architecture, we evaluate two prevalent design paradigms: memory-based implementations and pipelined implementations. We adopt the pipelined architecture for the following reasons: our design aims to decouple the computation rates of the PWM and NTT to enable parallel computation. Simply increasing the resources of the PWM leads to excessive hardware consumption. Therefore, we reduce the PWM computation rate. However, reducing the NTT's computation rate increases its processing latency. Thus, we instead reduce the NTT's output rate, that is, the number of outputs generated per cycle.

In summary, memory-based implementations have output rates that match their computation rates. Designing a standalone PWM module requires instantiating N PWM units (where N is the number of BFUs) to process NTT outputs, which wastes hardware resources. Reducing the number of BFUs to match PWM units prolongs NTT computation, increasing overall latency. Conversely, pipelined NTT architectures have output rates significantly lower than computation rates, allowing for optimal PWM quantity configuration and reduced hardware consumption.

However, pipelined architectures introduce a major drawback: pipeline buffering cycles. For example, processing a 256-term polynomial with the R2MDC NTT structure results in a total latency of 379 clock cycles, with pipeline buffering accounting for 66.2% of this time. In the Dilithium algorithm, serial input of multiple polynomials to the NTT reduces the

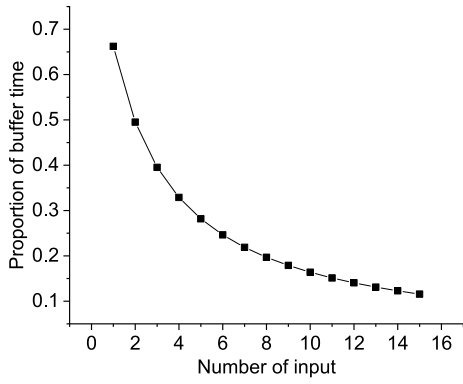


Fig. 4. Proportion of pipeline buffering cycles within the total NTT computation cycles. The proportion decreases as the security level increases, reaching 11.5% at Security Level V, indicating that the cycle overhead of the MDC structure becomes progressively less significant at higher security levels.

relative buffering overhead. Higher security levels require processing more polynomials, but the pipeline buffering latency remains constant at 251 clock cycles. Hence, the buffering proportion is

$$\text{Buffering Proportion} = \frac{251}{128 \times (k + l) + 251}$$

indicating that as security levels increase, the relative overhead of buffering decreases, demonstrating the practical viability of this architecture. This behavior is illustrated in Fig. 4.

Therefore, although the conventional MDC NTT architecture requires slightly more computation cycles than memory-based methods, our customized PWM module enables parallel execution of polynomial multiplication with NTT/INTT operations, reducing clock cycle overhead by 30%–40% compared to traditional memory architectures. Furthermore, while memory-based designs require additional storage and complex addressing, the proposed MDC NTT architecture offers significant advantages in both temporal efficiency and hardware resource utilization, as shown in Fig. 1.

1) *Top-Level Structure of Polynomial Multipliers*: The top-level architecture of our design is illustrated in Fig. 5 and comprises two main components: NTT/INTT and PWM, which completes the polynomial multiplication.

Based on the pipelined R2MDC NTT architecture, we designed a dedicated polynomial multiplication unit comprising four PEs. After the first two datasets complete eight rounds of NTT, two of the PEs directly perform polynomial multiplication with previously computed data. When the NTT output needs to be multiplied with multiple data sets—such as during the sign phase where the output c' is multiplied separately with s_1 , s_2 , and t_0 —all four PEs are utilized. This design enables dynamic throughput matching, eliminates the need for buffering NTT outputs, and significantly reduces computational latency.

2) *NTT/INTT*: The NTT module, depicted in Fig. 5, integrates pipeline registers within the BF and sort units to mitigate the frequency bottlenecks inherent in traditional R2MDC designs. For data buffering, a hybrid storage strategy is employed to optimize resource allocation. Since the buffering depths across the eight NTT rounds decrease geometrically from 64 to 1, BRAMs are utilized for the deeper Sort1 and Sort2 stages, while registers are allocated for the shallower

Sort3 through Sort7 stages. This depth-aware configuration maximizes resource efficiency.

3) *BF Unit*: The five-stage pipelined BF unit (Fig. 6) is designed to support both GS and CT configurations, enabling NTT and INTT execution at high frequencies. Modular arithmetic is implemented via Montgomery reduction [26]. To optimize resource usage, multiplications by the fixed Montgomery constants (5 872 8449 and 83 804 17) are converted into shift-and-add operations. Specifically, to minimize the number of required adders, we exploit that $83\,804\,16 + 8\,192 = 83\,886\,08$ (where $83\,886\,08 = 2^{23}$), which has a single “1” bit in binary. The same optimization applies to the constant 5 872 8449, further saving hardware resources.

Furthermore, the 28×23 -bit multiplication required prior to reduction exceeds the native 25×18 -bit capacity of Xilinx 7-series DSP slices. To prevent automatic DSP cascading, which degrades operating frequency, we explicitly decompose the operation into parallel 11×23 -bit and 17×13 -bit multiplications [Fig. 6(c)]. Although this manual partitioning slightly increases logic usage, it yields substantial timing improvements. Additionally, shift-and-add optimizations are selectively applied to the fixed twiddle factors in BF1 and BF2 to save DSP resources, whereas standard multipliers are retained for subsequent stages due to the large volume of distinct factors.

4) *Multiplication Calculation Example*: Fig. 7 exemplifies the vector–matrix multiplication workflow under Security Level II. To synchronize with the NTT output throughput, two PEs are activated to process the first two matrix rows in parallel. Valid output from the final BF stage (BF8) emerges after 132 cycles, triggering immediate polynomial multiplication. By cycle 686, the first two rows are processed, and computation transitions to the subsequent rows. Crucially, a synchronization delay is introduced prior to the dot product operation. Since the INTT processing rate exceeds that of the polynomial multiplication, this delay compensates for the rate mismatch, thereby preventing data interruptions and ensuring seamless pipeline continuity.

C. Coefficient Generator

The CoeGen is pivotal across key generation, signing, and verification phases, responsible for deriving matrix $\mathbf{A}$ and vectors s_1, s_2, y , and w . Software profiling (Fig. 8) reveals that CoeGen dominates the computation, accounting for approximately 55% of the total execution time during key generation, thereby necessitating a high-performance hardware implementation.

To ensure a compact and complete hardware implementation, we adopt a straightforward system architecture based on the foundational works of [11] and [12]. Specifically, we utilize the unified SHAKE128/256 structure [12] to support multimode operation. However, our specific implementation choices are driven by the system-level “Throughput-Matching” constraint.

We deliberately chose the 24-cycle per round scheme [11] over the faster 12-cycle alternative. While a 12-cycle core offers higher raw throughput, it would impose excessive pressure on the downstream sampling module (>4.6 data sets/cycle), necessitating complex and resource-heavy parallel

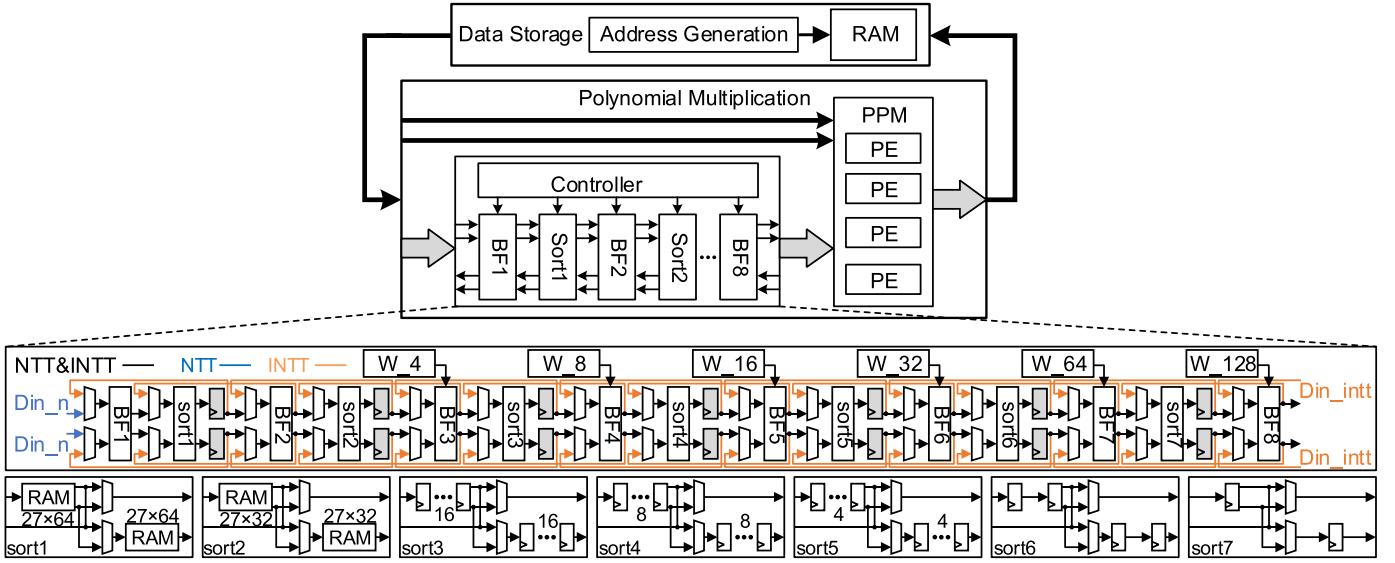


Fig. 5. Structure of fully pipelined R2MDC module.

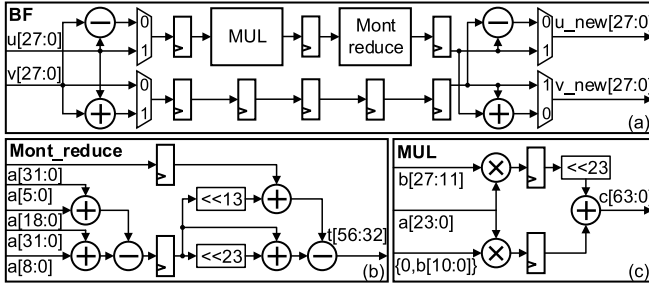


Fig. 6. Five-stage pipelined low-area BF structure.

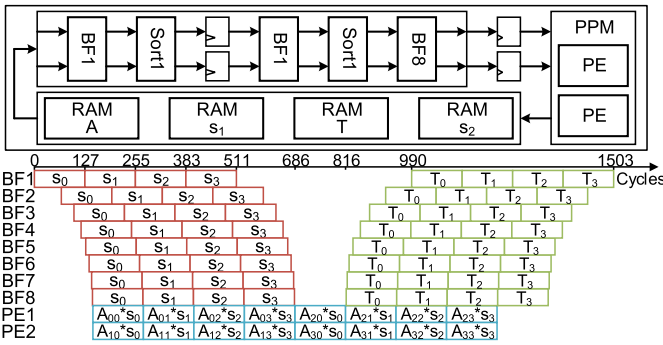


Fig. 7. Illustration of the vector-matrix multiplication process for Security Level II, showing the timing relationship between the NTT pipeline and the polynomial multiplication unit. The first data batch enters BF1 at cycle 0, with BF8 producing its first output at cycle 132, enabling the parallel computation of the first and second matrix rows using two PEs. By cycle 686, the first and second rows are complete, and computation proceeds to the third and fourth rows.

samplers. The 24-cycle configuration provides a data rate that perfectly matches our pipelined sampler, achieving the optimal balance between area and throughput.

To further optimize this standard core, we analyzed the Keccak round constants (RC) and identified that 75% are zero. We pruned the corresponding redundant XOR operations (Table II), reducing the logic footprint beyond standard implementations.

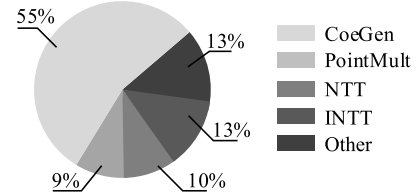


Fig. 8. Dilithium's time share for each part of the software.

TABLE I
DILITHIUM PARAMETER SET

Parameter	NIST Security Level		
	II	III	V
q[modulus]	8380417		
n[degree of polynomial]	256		
d[dropped bits from t]	13		
Eta[s ₁ /s ₂ sample]	2/4		
γ ₁ [y coefficient range]	2 ¹⁷	2 ¹⁹	2 ¹⁷
η[secret key range]	2	4	2
(k,ℓ)[dimensions of A]	(4,4)	(6,5)	(8,7)

For vector $\mathbf{y}$, we mitigate the 64-bit to 18-bit width mismatch via an LCM-based partitioning strategy (576 bits). By processing fixed 9×64 -bit blocks, this approach enables continuous data flow with minimal buffering, significantly reducing the area and latency overhead associated with full intermediate storage.

1) *Top-Level Architecture of CoeGen*: The unified CoeGen architecture, illustrated in Fig. 9, supports all three Dilithium security levels. It comprises five functional stages: a pseudo-random number generator (PRNG), a SHAKE128/256 core, a data preparation unit, an FSM, and a sampling module.

The process initiates in the PRNG, which employs two parallel 32-bit linear feedback shift registers (LFSRs). By concatenating their outputs, the module achieves a 64-bit throughput, generating the requisite 256-bit initial seed within

TABLE II
OPTIMIZATION OF RC[I]

RC[i]	Optimized RC[i]	RC[i]	Optimized RC[i]
0X0000000000000001	7'b0000001	0X000000008000808B	7'b0111111
0X0000000000008082	7'b0011010	0X800000000000008B	7'b1001111
0X800000000000808A	7'b1011110	0X8000000000008089	7'b1011101
0X800000000000808A	7'b1110000	0X8000000000008003	7'b1010011
0X000000000000808B	7'b0011111	0X8000000000008002	7'b1010010
0X0000000080000001	7'b0100001	0X8000000000000080	7'b1001000
0X8000000080008081	7'b1111001	0X000000000000800A	7'b0010110
0X8000000000008009	7'b1010101	0X800000008000000A	7'b1100110
0X000000000000008A	7'b0001110	0X8000000080008081	7'b1111001
0X0000000000000088	7'b0001100	0X8000000000008080	7'b1011000
0X0000000080008009	7'b0110101	0X0000000080000001	7'b0100001
0X000000008000000A	7'b0100110	0X8000000080008008	7'b1110100

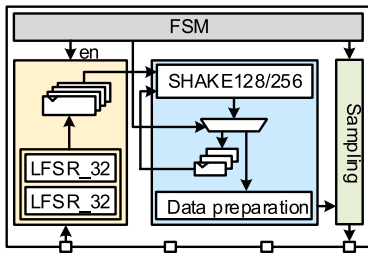


Fig. 9. CoeGen architecture.

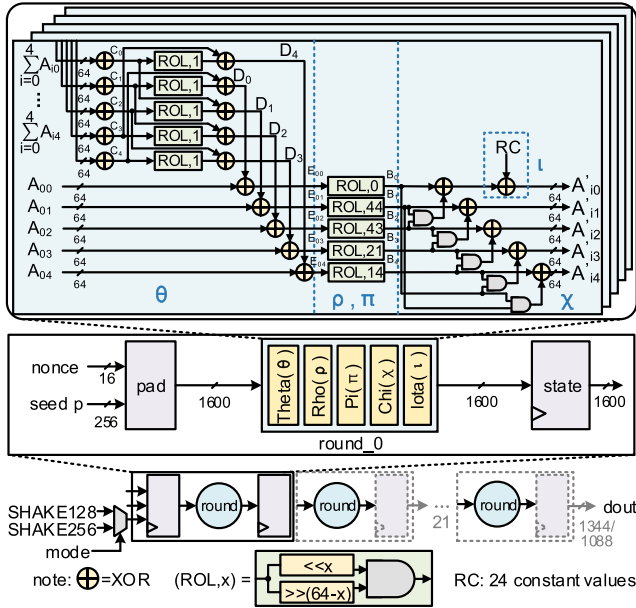


Fig. 10. Data flow and architecture of SHAKE128/256.

four clock cycles. This seed drives the SHAKE core, which first operates in SHAKE256 mode. From the initial output, the middle 32 bytes of the first 96 bytes are extracted to seed the subsequent SHAKE128 operation. The data preparation unit then adapts the 64-bit SHAKE output to align with the 24-bit input width required downstream. Finally, the sampling module executes *ETA* and *rejection sampling* sequentially to derive the coefficients for vectors $\mathbf{s}_1$, $\mathbf{s}_2$, and matrix $\mathbf{A}$.

2) *SHAKE128/256 and Sampling*: The SHAKE128/256 module (Fig. 10) features a unified architecture supporting dual modes via configurable (r, c) parameters ($r = 1344, c = 256$ for SHAKE128; $r = 1088, c = 512$ for SHAKE256). Despite differing valid output lengths, both modes execute 24 identical permutation rounds comprising the standard sequence: θ , ρ , π , χ , and ι .

The sampling module is employed to filter the output of the SHAKE128/256 generator to ensure data validity. During the generation of matrix $\mathbf{A}$ and vectors $\mathbf{s}_1$ and $\mathbf{s}_2$, the input data is zero-padded by extending the most significant bit. Upon successful sampling, the MSB of the qualified data is asserted (set to 1). Subsequently, the validated data is transmitted to the storage module with the MSB concatenated. This transmission strategy, based on concatenation, optimizes storage efficiency and reduces BRAM resource consumption by a factor of at least two. Specifically, for matrix $\mathbf{A}$, four parallel 24-bit input streams are filtered, and valid samples are buffered in temporary registers until a full 48-bit word is assembled for downstream storage.

A specific challenge arises during vector $\mathbf{y}$ generation due to the bit-width mismatch between the 64-bit Keccak output and the target 18-bit coefficients. Storing the entire intermediate output (85×64 bits) would impose significant latency and register overhead. To address this, we partition the data processing based on the least common multiple of 64 and 18, which is 576 bits. This alignment allows the data to be processed in fixed 9×64 -bit blocks. Consequently, the splitting operation is completed using only two hardware units with a minimal internal buffer, eliminating the need for bulky intermediate storage and redundant dismantling cycles.

IV. EXPERIMENTAL RESULTS AND COMPARISON

The proposed architecture was implemented in Verilog and synthesized using Vivado 2019.2 on the Xilinx Zynq-7000 system-on-chip (SoC). We present comprehensive resource utilization and performance metrics across all three NIST security levels. Furthermore, to facilitate equitable benchmarking against prior art, implementation results for Artix-7, Kintex-7, and Virtex Ultrascale+ platforms are included.

A. Implementation Results

Table III details the hardware resource utilization for the critical components of the proposed architecture. With the escalation of security levels, the corresponding expansion in vector dimensions necessitates increased computational cycles and larger storage capacity, whereas the resource footprint of the remaining components remains constant.

The polynomial multiplier exhibits a constant footprint across all security levels, consuming 5837 LUTs, 4304 FFs, 44 DSPs, and four BRAMs. The specific DSP allocation is attributed to its dual functionality: NTT/INTT utilizes 12 DSPs for twiddle factor multiplication in rounds 3–8, while the dedicated PWM unit employs 32 DSPs (distributed as eight DSPs per PE). Additionally, the four BRAMs are dedicated to storing twiddle factors for the final three NTT rounds.

The CoeGen module, comprising the unified SHAKE128/256 core and the sampling unit, utilizes 6486

TABLE III

RESOURCE UTILIZATION OF THE HARDWARE MODULE FOR DILITHIUM.
NOTATION FOR PLATFORM: ZYNQ-7000 (Z7)

Module	Security Level	LUT	FF	DSP	BRAM
PM	II/III/V	5837	4304	44	4
[NTT]	II/III/V	4049	3195	12	4
CoeGen	II/III/V	6486	4192	0	0
[SHAKE128/256]	II/III/V	3255	1643	0	0
[Sample]	II/III/V	3231	2549	0	0
Other	II	5033	3433	0	14
	III	5458	4133	0	18
	V	6656	5290	0	22
Total	II	18340	12297	44	18
	III	18765	13097	44	22
	V	19663	14154	44	26

TABLE IV

ASIC POWER AND ENERGY ESTIMATION FOR THREE SECURITY LEVELS
OF DILITHIUM (SMIC 55 NM, 166.6 MHz)

Security Level	Power (mW)	t(cycles)	Energy (μ J)
Dilithium-II	59.23	28.4	10.09
Dilithium-III	64.89	37.7	14.68
Dilithium-V	71.92	49.3	21.27

Note: Energy = Power $\times$ Cycles $\times$ Clock Period.

LUTs and 4192 FFs. This utilization reflects the complexity of supporting dual SHAKE modes and integrating both rejection and ETA sampling mechanisms. In contrast, the resource usage of the other modules (including FSM and data storage) scales with the security level. Specifically, BRAM consumption increases to 14, 18, and 22 blocks for levels II, III, and V, respectively, necessitated by the growth in vector and matrix dimensions.

To provide a comprehensive evaluation of the proposed architecture, we performed an ASIC-based power estimation. The design was synthesized using the SMIC 55nm low-leakage (LL) high-density (HD) standard cell library via Synopsys Design Compiler. To evaluate the design under stringent operating conditions, the physical setup was constrained at the slow-slow (SS) corner with a supply voltage of 1.08 V and an operating temperature of 125 °C.

The design was synthesized at an operating frequency of 166.6 MHz (clock period of 6.0 ns). The dynamic power calculation was performed using the tool's default switching activity propagation. The estimated power consumption and the corresponding energy required per operation across different security levels are summarized in Table IV. Furthermore, since total energy consumption is directly proportional to execution latency, the significant reduction in computation cycles achieved by our proposed decoupled architecture translates directly into a highly optimized energy footprint.

B. Comparison With the State-of-the-Art Works

Table V compares the NTT/INTT modules of our design against related works. To ensure equitable benchmarking, we employ the ATP metric. Resource utilization is normalized to equivalent slices (ENS) via the formula: $ENS = Slices + 100 \times DSP + 196 \times BRAM$, where

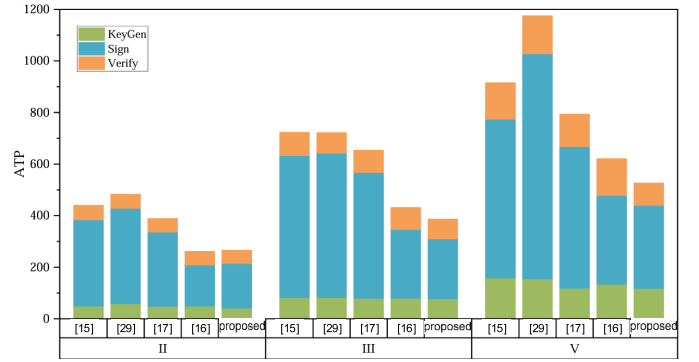


Fig. 11. ATP comparison across three stages under three security levels. At Security Level III, our ATP is comparable to that of [16], while at Security Level V, our design achieves the smallest ATP among all compared implementations.

slices are defined as LUT/4 (7-series) or LUT/8 (UltraScale+) [29], [30].

Zhao et al. [14] utilized a folding transformation to merge adjacent BFUs. While this halved DSP usage, it increased latency and limited operating frequency due to a lack of internal optimization. Wang et al. [12] adopted a Radix-4 NTT structure with a single BFU. However, limited pipeline depth restricted their frequency to 217 MHz (versus our 270 MHz). Although their compact design offers a lower ATP at Security Level II, our higher frequency yields superior ATP at Levels III and V. Li et al. [28] employed a memory-based architecture with internal BF pipelining; however, extensive BRAM usage and a lower frequency (220 MHz) resulted in a higher ATP.

Cui et al. [17] proposed an optimized modular multiplication algorithm integrated with a fully pipelined Radix-2 MDC-NTT architecture, which streamlines the overall hardware design and mitigates the resource overhead of the NTT module. To further reduce computation latency, their design employs dual parallel NTT units. Although the fully pipelined structure enhances the operating frequency, the aggregate computation latency remains relatively high. Furthermore, [17] lacks explicit detail regarding the utilization of BRAM for twiddle factor storage within the NTT module, reporting only the system-level BRAM usage. Consequently, the NTT component in their implementation exhibits reduced Slice utilization, yielding a lower ATP.

In contrast, our NTT design prioritizes computational speed, achieving the lowest ATP on Z7, K7, and VUS+ platforms (Table V), with the A7 platform being the sole exception where [17] performs marginally better. However, at the system level, our integration of dedicated point-wise multiplication modules and parallelized dataflow significantly reduces global cycle counts, ensuring superior overall execution time and ATP.

Table VI benchmarks the proposed comprehensive Dilithium hardware implementation against state-of-the-art designs. To facilitate a clearer visualization of ATP, Fig. 11 presents bar charts contrasting these metrics across the three distinct cryptographic phases as well as the overall system performance, evaluated on the A7 FPGA platform.

Li et al. [15] aggregate hardware resources to accelerate the sampling and polynomial multiplication stages to reduce latency. Specifically, their sampling module occupies a larger area to achieve higher throughput. Consequently, in

TABLE V
FPGA IMPLEMENTATION RESULTS OF NTT/INTT MODULE. NOTATION FOR PLATFORM: ZYNQ-7000 (Z7),
ARTIX-7 (A7), KINTEX 7 (K7), VUS+ (ULTRASCALE+)

Work	Security Level	Platform	Butterfly arrangement	LUT	FF	DSP	BRAM	ENS	Freq(MHz)	t(cycles)	Latency(us)	ATP(K)
ICFPT'21 [10]	II/III/V	K7	2×2	9018	6292	16	- ¹	4641	256	1024/1408/1920	4/5.5/7.5	18.5/25.5/32.4
ARES'21 [11]	II/III/V	VUS+	2×2	2236	2532	48	3.5	5937	637	4096/5632/7680	6.4/8.8/12.0	38.1/52.4/71.5
TVLSI'22 [12]	II/III/V	Z7	Radix-4	2386	2539	8	1	1744	217	2112/2904/3960	9.7/13.3/18.2	16.9/23.3/31.8
IACR'22 [14]	II/III/V	A7	Radix-2 MDC	1919	1301	8	2	1834	99.8 ²	2640/3408/4463	26.4/34.1/44.7	48.7/62.9/82.4
DAC'22 [27]	II/III/V	A7	Arbitrary-radix	1256	945	4	2.5	1333	204	-/5896/-	-/38.5/-	-/-/-
TVLSI'24 [28]	II/III/V	VUS+	Radix-2	11006	8791	12	24	6351	220	1248/1716/2340	5.6/7.8/10.6	35.5/49.5/67.3
TC'25 [17]	II/III/V	A7	Radix-2 MDC	4187	4387	18	- ¹	2847	297	2288/3146/4290	7.7/10.5/14.4	21.9/29.8/40.9
Proposed	II/III/V	A7	Radix-2 MDC	4040	3195	12	4	3170	151	1372/1756/2268	9.1/11.6/15.1	<u>28.8/36.8/47.6</u>
		Z7	Radix-2 MDC	4049	3195	12	4	3242	270	1372/1756/2268	5.1/6.5/8.4	<u>17.2/22.0/28.4</u>
		VUS+	Radix-2 MDC	4132	2703	12	4	3150	487	1372/1756/2268	2.8/3.6/4.6	8.8/11.3/14.6
		K7	Radix-2 MDC	4049	3195	12	4	3150	285	1372/1756/2268	4.8/6.1/7.9	15.1/21.0/27.1

1. The resource consumptions are absent from [10] and [17]. 2. The overall frequency is used because the frequency of the module is not provided in [14]

TABLE VI
FPGA IMPLEMENTATION RESULTS OF DILITHIUM. NOTATION FOR PLATFORM: ARTIX-7 (A7), ZYNQ-7000 (Z7)

Work	Platform	Serity Level	LUT	FF	BRAM	DSP	ENS	Freq(MHz)	t(cycles)	Latency(us)	ATP($\times 10^4$)
TVLSI'22 [12]	Z7	II	18558	7342	17	10	9960	159	7.7/52.0/7.6	49/327/48	<u>48.8/325.7/47.8</u>
		III	19614	8466	21	10	11961	159	12.9/89.2/11.2	82/561/71	98.1/671.0/84.9
		V	20973	9677	28	10	13052	159	20.1/93.7/15.8	127/589/100	165.8/768.8/130.5
CHES'22 [14]	A7	II	29998	10336	11	10	11991	96.9	4.1/28/4.4	<u>43/290/46</u>	51.5/335.7/55.1
		III	29998	10336	11	10	11991	96.9	5.8/44/6.1	<u>60/461/64</u>	<u>71.9/552.7/76.7</u>
		V	29998	10336	11	10	11991	96.9	8.7/49/9	<u>90/506/93</u>	<u>107.9/606.7/111.5</u>
ACISP'24 [31]	A7	II	26521	8144	16	8	12668	114.6	5.2/33/4.9	45.8/292.5/43	58.0/370.5/54.5
		III	23378	9079	20	8	11418	109	7.8/54/7.6	72.2/490.6/69.7	82.4/560.2/79.6
		V	28791	11338	24	8	14214	114.3	12.4/72/11.8	109.2/631.7/103.4	155.2/872.3/147.0
TCASI'23 [32]	A7	V	13975	6845	35	4	12116	<u>163</u>	63.2/113.9/67.9	387/699/416	468.2/846.9/504.1
TCASII'24 [15]	A7	II	32320	9734	14	24	14496	130	4.4/30.1/5.0	33.9/231/38.5	49.1/334.8/55.8
		III	32320	9734	14	24	14496	130	7.3/49.5/8.0	56.1/381/61.6	81.3/552.2/89.2
		V	32320	9734	14	24	14496	130	10.9/55.2/12.7	83.8/425/97.6	158.0/616.1/141.4
HOST'24 [16]	A7	II	30323	13585	28	16	14780	119	3.9/12.8/4.2	<u>33/108/36</u>	48.7/159.6/53.2
		III	30323	13585	28	16	14780	119	6.5/21.4/6.9	<u>54/180/58</u>	<u>79.8/266/85.7</u>
		V	30323	13585	28	16	14780	119	10.9/27.6/11.5	<u>91/232/97</u>	134.5/342.9/143.3
TC'25 [17]	A7	II							4.8/28.8/5.2	25.1/150.1/27.2	<u>48.1/287.8/52.1</u>
		III	52888	31213	11	38	19178	191	7.9/48.6/8.5	41.4/254.5/44.5	<u>79.3/488/85.9</u>
		V							11.8/54.7/12.5	61.8/286.4/65.4	<u>118.5/549.2/125.4</u>
Proposed	A7	II	18447	12297	18	44	12539	134	4.4/18.5/5.5	<u>32.8/138/41.1</u>	41.1/173/51.5
		III	19038	13097	22	44	13471	134	7.7/23.1/6.9	<u>57.4/172.3/57.3</u>	77.3/232.1/77.1
		V	19463	14154	26	44	14361	134	11.1/30.1/8.1	<u>82.1/224/60.4</u>	117.9/321.6/86.7
Proposed	Z7	II	18340	12297	18	44	14102	168	4.4/18.5/5.5	26.1/110.1/32.7	36.8/155.1/46.1
		III	18765	13097	22	44	15196	168	7.7/23.1/6.9	45.8/137.6/41.1	69.5/209.1/62.4
		V	19663	14154	26	44	16234	168	11.1/30.1/8.1	66.1/179/48.2	107.3/290.5/78.4

the CoeGen stage at Security Level III, their design achieves a slightly better ATP than ours. However, due to our optimized pipeline and resource management, our design demonstrates a lower ATP across all other security levels and stages.

Wu et al. [31] dynamically compute the polynomial matrix $\mathbf{A}$, thereby eliminating the need for static storage. This strategy reduced BRAM usage to 16, 20, and 24 across the three security levels, respectively—two fewer blocks than required by our design. However, this memory efficiency comes at the cost of logic complexity. Our architecture consumes fewer LUTs, which counterbalances the higher DSP

usage inherent to our dedicated multipliers, resulting in a comparable overall ENS performance.

Cui et al. [17] adopted a parallel strategy, deploying two NTT-based multipliers and three Keccak cores to accelerate the CoeGen phase. While this reduces the coefficient generation time, the execution times for the sign and verify phases remain largely unchanged. Furthermore, the significant hardware overhead associated with instantiating multiple cores leads to a higher overall ATP compared to our solution.

Zhao et al. [16] introduced a hybrid architecture replacing standard NTT-based multiplication with a sparse

multiplication core. This sparse core supports concurrent processing of multiple sparse polynomial multiplications, significantly shortening computation cycles in the sign and verify phases. As a result, at Security Levels II and III, their design achieves lower total execution time and ATP. However, the advantage of our dedicated polynomial multiplier lies in its scalability. At Security Level V, where computational demands peak, our design outperforms [16] and all other existing works in both execution time and ATP, utilizing 35% fewer LUTs while maintaining comparable ENS due to DSP substitution.

Finally, on the Zynq-7000 platform, our design achieves optimal performance in terms of frequency, execution time, and ATP among existing studies. Although employing a dedicated point-wise multiplication module increases DSP and ENS consumption compared to single-unit designs like [12], this tradeoff is justified. The resulting reduction in computation cycles and the increase in operating frequency ultimately yield the most favorable ATP results.

V. CONCLUSION

We propose a high-speed Dilithium hardware architecture featuring a polynomial multiplier with dynamic throughput matching. By employing independent polynomial multiplication units, the design aligns the throughput of NTT outputs with subsequent polynomial multiplication operations, enabling multistep parallelism. This approach significantly reduces intermediate data buffering, thereby saving substantial computation time. Across all three security levels, the proposed Dilithium architecture demonstrates superior performance relative to state-of-the-art designs, achieving an optimal ATP.

VI. FUTURE WORK

Future work will focus on implementing higher radix pipelined NTT structures to further boost throughput and reduce latency. Furthermore, we intend to extend our Dilithium-specific optimizations to develop a unified, reconfigurable polynomial multiplier capable of supporting Kyber and FALCON, achieving a versatile hardware solution for the postquantum era.

REFERENCES

- [1] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proc., 35th Annu. Symp. Found. Comput. Sci.*, 1994, pp. 124–134.
- [2] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Rev.*, vol. 41, no. 2, pp. 303–332, 1999.
- [3] L. O'Connor et al., "Cryptography in a postquantum world: Preparing intelligent enterprises now for a secure future," Accenture Labs, Dublin, Ireland, 2018.
- [4] L. Chen et al., "Report on post-quantum cryptography," U.S. Dept. Commerce, Nat. Inst. Standards Technol., USA, 2016, vol. 12.
- [5] G. Alagic et al., "Status report on the third round of the NIST post-quantum cryptography standardization process," National Institute of Standards and Technology, USA, Tech. Rep. 8413, 2022.
- [6] G. Alagic et al., "Status report on the fourth round of the NIST post-quantum cryptography standardization process," National Institute of Standards and Technology, USA, Tech. Rep. 8545, 2025.
- [7] G. Du, Z. P. Chen, Z. Li, X. Wang, and D. Zhang, "A low-latency polynomial multiplier accelerator for CRYSTALS-dilithium digital signature," in *Proc. Great Lakes Symp. VLSI*, 2024, pp. 258–262.
- [8] A. Langlois and D. Stehlé, "Worst-case to average-case reductions for module lattices," *Designs, Codes Cryptogr.*, vol. 75, no. 3, pp. 565–599, Jun. 2015.
- [9] M. Ajtai, "Generating hard instances of the short basis problem," in *Proc. 26th Int. Colloq., Automata, Lang. Program.* Cham, Switzerland: Springer, 1999, pp. 1–9.
- [10] L. Beckwith, D. T. Nguyen, and K. Gaj, "High-performance hardware implementation of CRYSTALS-Dilithium," in *Proc. Int. Conf. Field-Program. Technol. (ICFPT)*, Dec. 2021, pp. 1–10.
- [11] S. Ricci et al., "Implementing CRYSTALS-Dilithium signature scheme on FPGAs," in *Proc. 16th Int. Conf. Availability, Rel. Secur.*, Aug. 2021, pp. 1–11.
- [12] T. Wang, C. Zhang, P. Cao, and D. Gu, "Efficient implementation of dilithium signature scheme on FPGA SoC platform," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 30, no. 9, pp. 1158–1171, Sep. 2022.
- [13] G. Mao et al., "High-performance and configurable SW/HW co-design of post-quantum signature CRYSTALS-dilithium," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 16, no. 3, pp. 1–28, Sep. 2023.
- [14] C. Zhao et al., "A compact and high-performance hardware architecture for CRYSTALS-dilithium," in *Proc. IACR Trans. Cryptograph. Hardw. Embedded Syst.*, Nov. 2021, pp. 270–295.
- [15] X. Li, J. Lu, D. Liu, A. Li, S. Yang, and T. Huang, "A high speed post-quantum crypto-processor for crystals-dilithium," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 1, pp. 435–439, Jan. 2023.
- [16] H. Zhao, C. Zhao, W. Zhu, B. Yang, S. Wei, and L. Liu, "Sparse polynomial multiplication-based high-performance hardware implementation for CRYSTALS-dilithium," in *Proc. IEEE Int. Symp. Hardw. Oriented Secur. Trust (HOST)*, May 2024, pp. 150–159.
- [17] Y. Cui, J. Zhong, B. Wang, T. Xu, C. Wang, and W. Liu, "High-performance hardware implementation of crystals-dilithium based on improved MDC-NTT," *IEEE Trans. Comput.*, vol. 74, no. 9, pp. 2896–2908, Sep. 2025.
- [18] M. Bisheh-Niasar, R. Azarderakhsh, and M. Mozaffari-Kermani, "High-speed NTT-based polynomial multiplication accelerator for post-quantum cryptography," in *Proc. IEEE 28th Symp. Comput. Arithmetic (ARITH)*, Jun. 2021, pp. 94–101.
- [19] M. Bisheh-Niasar, R. Azarderakhsh, and M. Mozaffari-Kermani, "A monolithic hardware implementation of Kyber: Comparing apples to apples in PQC candidates," in *Proc. Int. Conf. Cryptol. Inf. Secur. Latin Amer. Bogotá, Colombia*: Springer, 2021, pp. 108–126.
- [20] D. Soni, K. Basu, M. Nabeel, N. Aaraj, M. Manzano, and R. Karri, "CRYSTALS-dilithium," in *Hardware Architectures for Post-Quantum Digital Signature Schemes*. Cham, Switzerland: Springer, 2021, pp. 13–30.
- [21] R. C. Agarwal and C. S. Burrus, "Number theoretic transforms to implement fast digital convolution," *Proc. IEEE*, vol. 63, no. 4, pp. 550–560, Apr. 2005.
- [22] Z. Liang and Y. Zhao, "Number theoretic transform and its applications in lattice-based cryptosystems: A survey," 2022, *arXiv:2211.13546*.
- [23] E. Chu and A. George, *Inside the FFT Black Box: Serial and Parallel Fast Fourier Transform Algorithms*. Boca Raton, FL, USA: CRC Press, 1999.
- [24] R. E. Hufnagel and N. R. Stanley, "An algorithm for the machine calculation of complex Fourier series," *J. Opt. Soc. Amer.*, vol. 54, no. 1, pp. 52–61, 1964.
- [25] W. M. Gentleman and G. Sande, "Fast Fourier transforms: For fun and profit," in *Proc. Amer. Fed. Inf. Process. Societies Joint Comput. Conf. (AFIPS)*, vol. 29, Nov. 1966, pp. 563–578.
- [26] M. Knezevic, F. Vercauteren, and I. Verbauwhede, "Faster interleaved modular multiplication based on Barrett and Montgomery reduction methods," *IEEE Trans. Comput.*, vol. 59, no. 12, pp. 1715–1721, Dec. 2010.
- [27] X. Chen, B. Yang, Y. Lu, S. Yin, S. Wei, and L. Liu, "Efficient access scheme for multi-bank based NTT architecture through conflict graph," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, Jul. 2022, pp. 91–96.
- [28] B. Li, Y. Yan, Y. Wei, and H. Han, "Scalable and parallel optimization of the number theoretic transform based on FPGA," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 32, no. 2, pp. 291–304, Feb. 2024.
- [29] *7 Series FPGAs Configurable Logic Block User Guide*, AMD Xilinx, Santa Clara, CA, USA, Apr. 2025.
- [30] *UltraScale Architecture Configurable Logic Block User Guide*, AMD Xilinx, Santa Clara, CA, USA, Jan. 2025.
- [31] Z. Wu, R. Chen, Y. Wang, Q. Wang, and W. Peng, "An efficient hardware implementation of CRYSTAL-dilithium on FPGA," in *Proc. Australas. Conf. Inf. Secur. Privacy*. Cham, Switzerland: Springer, 2024, pp. 64–83.
- [32] N. Gupta, A. Jati, A. Chattopadhyay, and G. Jha, "Lightweight hardware accelerator for post-quantum digital signature CRYSTALS-dilithium," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 8, pp. 3234–3243, Aug. 2023.